

Efficient LLM Edge Collaboration Deployment with LoRA

YangXiao, 1025040812

Nanjing University of Posts and Telecommunications

School of Computer Science

Nanjing, China

email: 1025040812@njupt.edu.cn

Abstract—In recent years, large language models (LLMs) have shown great potential in many fields. LLMs deployed in cloud data centers are increasingly unable to meet the low-latency inference requirements of massive mobile users. Benefiting from various LLM lightweighting techniques and the continuously improving performance of edge servers, deploying LLMs on edge servers closer to mobile users and executing inference tasks locally can effectively reduce inference latency. However, edge servers have limited storage capacity, and deploying LLMs on edge servers incurs additional deployment overhead. In this paper, we propose an efficient LLM edge collaboration deployment strategy called EdgeColl, aiming to jointly optimize inference latency and LLM deployment costs. Specifically, EdgeColl adopts Low-Rank Adaptation (LoRA) to divide each LLM into a base model and a LoRA matrix. We formulate the LLM edge collaboration deployment problem with LoRA. Then, we present the base model deployment (BMD) strategy to achieve low inference latency and deployment costs. The LoRA deployment (LMD) strategy is also proposed to enable personalized inference. We evaluate the performance of EdgeColl. The experimental results show that EdgeColl effectively reduces LLM inference latency and deployment costs.

Index Terms—LLM deployment; Edge computing; LoRA

I. INTRODUCTION

In recent years, Large Language Models (LLMs) have received widespread attention, especially in the fields of natural language processing [1], healthcare [2], and autonomous driving [3]. Common LLMs such as Llama [4] and Qwen [5] are built on the transformer architecture and provide general-purpose inference services [6]. These LLMs are typically called base models. To meet domain-specific inference requirements, parameter-efficient fine-tuning techniques, such as Low-Rank Adaptation (LoRA) [7], can be employed to optimize base models, producing personalized LLMs that are better suited to target application scenarios [8].

At present, the vast majority of personalized LLMs are deployed in cloud data centers, leveraging their abundant computational and storage resources to support large-scale inference services [9]. However, the long physical distance between mobile users and the remote cloud often results in high end-to-end latency and unpredictable network jitter, making the inference latency undesirable for latency-sensitive applications [9]. Besides, transmitting user inference requests or intermediate results to the cloud raises privacy and data security concerns, particularly when dealing with sensitive

or personal information [6]. On this basis, numerous LLM lightweighting techniques have been proposed to deploy personalized LLMs on terminal devices. However, the computing, storage, and battery power of terminal devices are limited [10], [11], which constrains the performance of LLMs [12].

Edge computing is an attractive paradigm to solve the problem mentioned above. Edge servers located close to mobile users are equipped with more computation and storage capacity than terminal devices, therefore can deploy larger LLMs to provide high-accuracy and low-latency inference services for mobile users [6]. Specifically, each edge server has a coverage area and can deploy specific LLMs to satisfy the diverse inference requirements of mobile users. The inference requests issued by mobile users can be uploaded to the edge servers that cover them (which we refer to as the local edge servers). Upon receiving an inference request, the edge server performs task inference through the deployed personalized LLMs and then returns the inference result.

Existing studies have explored deploying LLMs at edge servers to achieve diverse optimization objectives [13], [14], [15], such as maximizing LLM hit ratio and minimizing inference latency. However, most studies overlook that different LLMs may share the same base model, leading to redundant cache during deployment. In fact, many LLMs are obtained by fine-tuning a few base models with LoRA [16]. The original parameters of the base model are frozen during fine-tuning, which means that there is substantial parameter overlap among different LLMs. Therefore, mobile users can reuse the parameters of base models cached on edge servers and combine them with different LoRA matrices to perform their personalized inference tasks. Moreover, in pursuit of low inference latency, existing studies often prioritize deploying as many LLMs as possible on edge servers, while ignoring the associated deployment overhead, which inevitably results from renting the edge servers. Therefore, balancing low deployment costs and low inference latency remains a key challenge in deploying LLMs at the edge.

In this paper, we investigate the LLM edge collaboration deployment problem. Specifically, to improve the storage utilization of edge servers and reduce redundant cache, we split LLMs into a set of base models and LoRA matrices. We formulate the LLM edge collaboration deployment problem with LoRA (LECPL) and jointly optimize inference latency

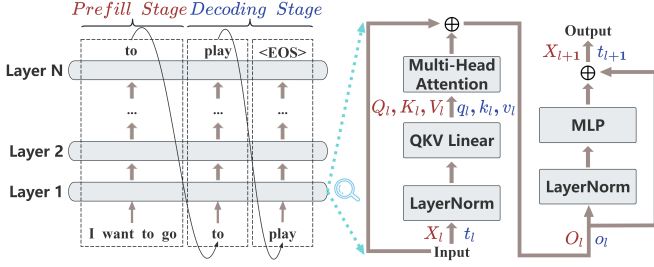


Fig. 1: LLM inference process

and LLM deployment costs. We then propose an efficient LLM edge collaboration deployment strategy named EdgeColl. EdgeColl decomposes LECPL into two subproblems, i.e., the base model deployment problem (BMD) and the LoRA matrix deployment problem (LMD). We first determine the locations of base models based on the distribution of historical user inference requests. Then, given the base model locations and the remaining storage capacity of edge servers, we determine the locations of LoRA matrices. When base models and LoRA matrices are not located on the same server, we construct a complete LLM through collaboration between edge servers to efficiently perform the inference tasks. The main contributions of this paper are as follows:

- (1) We develop a novel LLM deployment architecture that integrates base models and LoRA matrices to achieve personalized inference.
- (2) We jointly optimize the inference latency and LLM deployment costs. On this basis, we present EdgeColl to sequentially deploy the base models and LoRA matrices at edge servers.
- (3) We conduct extensive experiments to evaluate the performance of EdgeColl. The results show that EdgeColl effectively reduces LLM inference latency and deployment costs.

II. BACKGROUND AND MOTIVATING EXAMPLE

In this section, we first introduce the LLM inference process and LoRA technology. Then, we adopt a motivating example to illustrate the LLM edge deployment problem.

A. Background

LLM Inference: Mainstream LLM inference adopts the decoder-only Transformer architecture, which is composed of a stack of N identical layers [17]. Fig. 1 illustrates the LLM inference process. At the beginning, the user's input undergoes three sequential steps: tokenization, embedding, and position encoding, to produce the formatted tokens. Inference then proceeds through two stages: prefill and decoding. In the prefill stage, the LLM processes all formatted tokens simultaneously, generating and caching Key-Value (KV) pairs for reuse. During the decoding stage, the LLM performs serial inference iteratively. In each iteration, LLM uses cached KV values to generate the next token and updates the cache with the newly produced KV pairs, continuing until a termination condition is triggered.

The generation of each token requires passing through the entire LLM. The two most critical computational modules in this process are Multi-Head Attention (MHA) and Feed-forward Neural Network (FNN). Let g_i denote the number of input tokens of mobile user u_i ; d_m and d_f denote the hidden dimension of Transformer and FNN, respectively. In the prefill stage, user input is processed and the first token is generated. The input of the l -th layer is denoted by $X_l \in \mathbb{R}^{g \times d_m}$. Then KV values of l -th layer are calculated as follows:

$$K_l = X_l W_l^K, \quad V_l = X_l W_l^V.$$

Subsequently, the first token can be calculated as follows:

$$\begin{aligned} Q_l &= X_l W_l^Q, \\ A_l &= \text{softmax}\left(\frac{Q_l K_l^T}{\sqrt{d_m}}\right) V_l, \quad O_l = A_l W_l^O + X_l, \\ X_{l+1} &= \text{f}_{act}(O_l W_l^1) W_l^2 + O_l, \end{aligned}$$

where $W_l^Q, W_l^K, W_l^V, W_l^O \in \mathbb{R}^{d_m \times d_m}$ are the l -th layer parameter matrices in MHA, $W_l^1 \in \mathbb{R}^{d_m \times d_f}$, $W_l^2 \in \mathbb{R}^{d_f \times d_m}$ are the l -th layer parameter matrices in FNN, and O_l and X_{l+1} are the output of MHA and FNN at the l -th layer, respectively. In the decoding stage, LLM generates tokens one by one through autoregression. Let $t_l \in \mathbb{R}^{1 \times d_m}$ denote the input token of l -th layer. The subsequent token can be calculated as follows:

$$\begin{aligned} q_l &= t_l W_l^Q, \quad k_l = t_l W_l^K, \quad v_l = t_l W_l^V, \\ K_l &\leftarrow \text{Concat}(K_l, k_l), \quad V_l \leftarrow \text{Concat}(V_l, v_l), \\ a_l &= \text{softmax}\left(\frac{q_l K_l^T}{\sqrt{d_m}}\right) V_l, \quad o_l = a_l W_l^O + t_l, \\ t_{l+1} &= \text{f}_{act}(o_l W_l^1) W_l^2 + o_l. \end{aligned}$$

In this way, the LLM can continuously generate inference results for mobile users.

Low-Rank Adaptation: LoRA is a parameter-efficient fine-tuning technique that enables a base model to adapt to specific downstream tasks with minimal training cost. During training, LoRA freezes all parameters of the base model and injects two low-rank matrices $A \in \mathbb{R}^{r \times d_m}$ and $B \in \mathbb{R}^{d_m \times r}$ ($r \ll d_m$) into the LLM (we collectively refer to them as LoRA matrix). By training the LoRA matrix, it completes the fine-tuning of the base model. LoRA can be applied to any parameter matrix in the Transformer, but is usually applied to W^Q and W^V . Since the rank of the LoRA matrix is small, it significantly reduces training cost while achieving performance comparable to full-parameter fine-tuning.

Fig. 2 demonstrates the application of LoRA matrix in the LLM inference process. For trained LoRA matrix, we can choose to store LoRA matrix and base model's parameter matrices separately. When performing inference tasks, the LoRA matrix can be dynamically loaded into the base model's parameter matrices. The process is illustrated as follows:

$$Q = XW^Q + XB^Q A^Q, \quad V = XW^V + XB^V A^V.$$

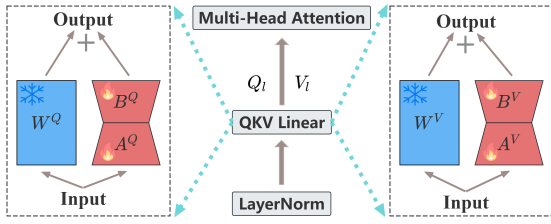


Fig. 2: LoRA in LLM inference process

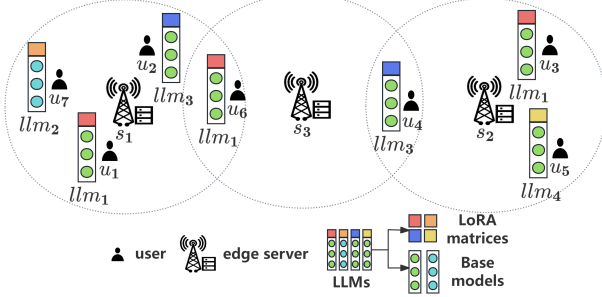


Fig. 3: An example scenario for LLM edge deployment

In the inference process, the overhead incurred by dynamic loading is negligible. It is precisely the dynamic loading of the LoRA matrices that enables EdgeColl.

B. A Motivating Example

In edge scenarios, users can send requests to local servers. Different users have diverse inference requirements, so it is necessary to deploy LLMs required by users on the local servers to provide services while ensuring inference accuracy.

Fig. 3 illustrates an example of LLM edge deployment, which includes three edge servers $\{s_1, s_2, s_3\}$, seven mobile users $\{u_1, u_2, u_3, u_4, u_5, u_6, u_7\}$, and four LLMs $\{llm_1, llm_2, llm_3, llm_4\}$. Each LLM consists of a base model and a LoRA matrix, i.e., $llm_1 = f_1 + r_1$, $llm_2 = f_2 + r_2$, $llm_3 = f_1 + r_3$, $llm_4 = f_1 + r_4$. Each user requests an LLM to meet their inference requirements. Each edge server covers a specific geographical area. By default, Mobile users upload inference requests to the local servers closest to them. We assume that each mobile user only sends one inference request, and each inference request is processed by one edge server. We set the size of each base model to 50GB, and the size of each LoRA matrix to 100MB. The storage capacity of each edge server are set to 120GB. We need to deploy LLMs on suitable edge servers, providing users with low-latency inference services while considering deployment costs.

However, edge servers have limited storage capacity, so we must propose a reasonable strategy to deploy LLMs. As shown in Fig. 3, u_3, u_4, u_5 within s_2 's coverage request llm_1, llm_3, llm_4 , respectively. The problem is that the storage capacity of s_2 can only deploy two LLMs. This means that one of the users only requests the LLM from the cloud. EdgeColl can effectively address this problem. Before deployment begins, EdgeColl splits the LLM into a base model and a LoRA matrix. Therefore, the components that actually need to be deployed are f_1, r_1, r_3 and r_4 , and this will not

exceed the storage capacity of s_2 . By dynamically loading different LoRA matrices and merging them with the base model f_1 , personalized inference services can be provided to users without uploading any user requests to the cloud.

In addition, we need to consider the deployment costs of LLMs, which have been largely overlooked in previous studies. In Fig. 3, we assume that each edge server has sufficient rentable storage capacity. In the previous deployment framework, which did not consider deployment costs, we first needed to rent three edge servers: s_1, s_2 , and s_3 in order to ensure that user requests could be processed at the edge. Then, to successfully deploy the required LLMs on these servers, a total of approximately 350.7 GB of storage would be required across the three servers. Under the EdgeColl deployment framework, we still rent three servers, s_1, s_2 , and s_3 . However, due to the elimination of redundant cache, the three servers only need a total of approximately 200.7GB, saving nearly half of the storage capacity. Moreover, we can only rent s_1 and s_2 , so that u_6 's request can be sent to s_1 . Although this incurs a small increase in communication latency, it reduces the costs of renting edge servers.

III. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we introduce the system model and formulate the LLM edge collaboration deployment problem with LoRA. The objective is to minimize the inference latency and deployment costs.

A. System Model

In the LLM edge deployment scenario, let $S = \{s_1, \dots, s_n\}$ denote the set of n edge servers. Each edge server s_j has its coverage area $cov(s_j)$ and storage capacity q_j . The set of m users is denoted by $U = \{u_1, \dots, u_m\}$. Each user may request different LLMs to perform their inference tasks. Let $LLM = \{llm_1, \dots, llm_k\}$ denote the set of k LLMs. To accelerate inference and protect user privacy, we deploy LLMs on edge servers close to users. Users can upload inference tasks to their local edge servers via a wireless link. To meet personalized inference requirements, we assume that each LLM consists of a base model and a LoRA matrix. Let $F = \{f_1, \dots, f_p\}$ and $R = \{r_1, \dots, r_q\}$ denote the set of base models and LoRA matrices, respectively. The size of base model f_l is denoted as D_l^{Base} , and the size of LoRA matrix r_h is denoted as D_h^{LoRA} . Since the storage capacity of edge servers is limited, it is impractical to deploy all LLMs on a single edge server. In addition, different LLMs may share identical base models. To ensure low-latency LLM inference and reduce model deployment overhead, we mainly focus on investigating the deployment of base models and LoRA matrices in edge servers.

Given the set of base models F and LoRA matrices R , the deployment strategy of base models can be expressed as $y = \{y_{j,l}, \forall s_j \in S, f_l \in F\}$, where $y_{j,l}$ indicates whether f_l is deployed on s_j . $y_{j,l}$ can be expressed as

$$y_{j,l} = \begin{cases} 1, & \text{if } f_l \text{ is deployed on } s_j, \\ 0, & \text{if } f_l \text{ is not deployed on } s_j. \end{cases} \quad (1)$$

Similarly, the deployment strategy of LoRA matrices can be expressed as $\mathbf{x} = \{x_{j,h}, \forall s_j \in S, r_h \in R\}$, where $x_{j,h}$ indicates whether LoRA matrix r_h is deployed on edge server s_j . $x_{j,h}$ can be expressed as

$$x_{j,h} = \begin{cases} 1, & \text{if } r_h \text{ is deployed on } s_j, \\ 0, & \text{if } r_h \text{ is not deployed on } s_j. \end{cases} \quad (2)$$

Since storage capacity on edge servers is limited, the total size of base models and LoRA matrices deployed on s_j should not exceed the storage capacity of s_j . Therefore, we have

$$\sum_{f_l \in F} D_l^{Base} y_{j,l} + \sum_{r_h \in R} D_h^{LoRA} x_{j,h} \leq q_j, \quad \forall s_j \in S. \quad (3)$$

We define $\delta_{j,i}$ to denote whether the inference request of u_i is uploaded to s_j . Therefore, we have

$$\delta_{j,i} = \begin{cases} 1, & \text{if request of } u_i \text{ is uploaded to } s_j, \\ 0, & \text{if request of } u_i \text{ is not uploaded to } s_j. \end{cases} \quad (4)$$

To efficiently utilize the resources of edge servers, we assume that a user's inference request can always be served by only one edge server. Therefore, we have

$$\sum_{s_j \in S} \delta_{j,i} = 1, \quad \forall u_i \in U. \quad (5)$$

Besides, the inference request of u_i is processed by s_j only if s_j covers u_i and has deployed the corresponding base model f_l . If local edge servers do not deploy the corresponding base model f_l , the inference request will be processed through the cloud. Therefore, we have

$$\delta_{j,i} = 0, \quad \forall u_i \notin \text{cov}(s_j), \quad (6)$$

$$\delta_{j,i} \leq y_{j,l}, \quad \forall u_i \in U, s_j \in S. \quad (7)$$

It is worth noting that we do not require LoRA matrix r_h and base model f_l to be deployed on the same edge server. When a user's inference request arrives, if local edge servers do not deploy the corresponding LoRA matrix, it can retrieve the LoRA matrix from another edge server.

B. Total Inference Latency

The total inference latency consists of (1) Communication latency. The latency of uploading user inference requests and downloading inference results; (2) LoRA matrix transmission latency. The latency of transferring LoRA matrix to the edge server hosting the base model when the two are deployed on different edge servers; (3) Task inference latency. The latency of model inference execution. Similar to [18], this paper employs a dynamic batching approach that enables parallel processing of user inference tasks without waiting for requests to reach a fixed batch size. Therefore, we ignore the queuing latency of inference tasks.

Communication latency. Let I_i denote the size of input inference task issued by u_i , and O_i denote the size of output inference results received by u_i . Mobile users upload inference tasks and download results via wireless links. According to

Shannon's formula, the upload rate $R_{j,i}^{up}$ and download rate $R_{j,i}^{dw}$ can be expressed as follows:

$$R_{j,i}^{up} = B_{up} \log_2 \left(1 + \frac{e_{u_i} \gamma_0 \text{dis}_{j,i}^{-\beta_0}}{n_0 B_{up}} \right), \quad (8)$$

$$R_{j,i}^{dw} = B_{dw} \log_2 \left(1 + \frac{e_{s_j} \gamma_0 \text{dis}_{j,i}^{-\beta_0}}{n_0 B_{dw}} \right), \quad (9)$$

where γ_0 is the antenna-related factor, β_0 is the path loss factor, n_0 is the power spectral density of the additional Gaussian white noise, $\text{dis}_{j,i}$ is the distance between user u_i and edge server s_j , e_{u_i} and e_{s_j} are the transmission powers of user u_i and edge server s_j respectively. B_{up} and B_{dw} are the uplink channel bandwidth and downlink channel bandwidth, respectively. Therefore, the communication latency between mobile user u_i and edge server s_j can be expressed as follows:

$$T_{i,j}^{comm} = \sum_{s_j \in S} \delta_{j,i} \left(\frac{I_i}{R_{j,i}^{up}} + \frac{O_i}{R_{j,i}^{dw}} \right) + (1 - \sum_{s_j \in S} \delta_{j,i} y_{j,l}) \left(\frac{I_i}{R^{cloud}} + \frac{O_i}{R^{cloud}} \right), \quad (10)$$

where R^{cloud} is the transmission rate of the backhaul link between the edge server and the cloud data center.

LoRA matrix transmission latency. If the base model requested by user u_i is deployed on server s_j while the corresponding LoRA matrix is deployed on server $s_{j'}$, the LoRA matrix needs to be transferred from $s_{j'}$ to s_j to complete the inference task. Assuming that servers communicate via wireless links, the transmission latency of the LoRA matrix can be expressed as:

$$T_{j,j'}^{LoRA} = \sum_{j \in \{j: y_{j,l}=1\}, j' \in \{j': x_{j',h}=1\}} \frac{D_h^{LoRA}}{R_{j,j'}}, \quad (11)$$

where $R_{j,j'}$ is the transmission rate between s_j and $s_{j'}$.

Task inference latency. Let $T_{i,j}^{inf}$ denote the task inference latency of user u_i at edge server s_j , which is constrained by computational power and memory bandwidth [19]. On one hand, during the prefill stage, Transformer performs numerous matrix multiplications over the entire input sequence, relying heavily on floating-point computing power. On the other hand, during the decoding stage, Transformer frequently accesses and updates the KV cache, placing high demands on memory bandwidth. We denote the latency limited by computing power as $T_{i,j}^{com}$, and that limited by memory bandwidth as $T_{i,j}^{mem}$. Hence, $T_{i,j}^{inf}$ is determined by the maximum of the two:

$$T_{i,j}^{inf} = \max(T_{i,j}^{com}, T_{i,j}^{mem}). \quad (12)$$

First, we calculate $T_{i,j}^{com}$ when computational power is the bottleneck. The key to computing $T_{i,j}^{com}$ lies in the statistics of computational load. Since the computational load of MHA and FNN accounts for the majority of inference process, the load from other operations, e.g., layer normalization, can be

ignored [20]. Therefore, when computing power is limited, the inference latency of the prefill stage and decoding stage is:

$$T_{i,j}^{com}(pre) = \frac{C_{i,j}^{pre}}{V_j}, \quad (13)$$

$$T_{i,j}^{com}(dec) = \frac{C_{i,j}^{dec}}{V_j}, \quad (14)$$

where $C_{i,j}^{pre}$ and $C_{i,j}^{dec}$ are respectively the computational load in the prefill stage and the decoding stage. V_j denotes the computing speed of server s_j , measured in $FLOPs/s$.

Next, we calculate $T_{i,j}^{mem}$ when memory bandwidth is the bottleneck. The key here lies in the statistics of memory access volume. The memory access volume during inference primarily arises from read/write operations on the KV cache. Since parameter matrices typically reside in memory and rarely need to be reloaded, their access volume can be neglected [19]. Therefore, when memory bandwidth is limited, the inference latency of the prefill stage and decoding stage is:

$$T_{i,j}^{mem}(pre) = \frac{M_{i,j}^{pre}}{BW_j}, \quad (15)$$

$$T_{i,j}^{mem}(dec) = \frac{M_{i,j}^{dec}}{BW_j}, \quad (16)$$

where $M_{i,j}^{pre}$ and $M_{i,j}^{dec}$ are respectively the memory access volumes in the prefill stage and the decoding stage. BW_j is the memory access bandwidth of server s_j .

Substituting (13)-(16) into (12), we obtain $T_{i,j}^{inf}$ as follows:

$$T_{i,j}^{inf} = \max(T_{i,j}^{com}(pre), T_{i,j}^{mem}(pre)) + \max(T_{i,j}^{com}(dec), T_{i,j}^{mem}(dec)). \quad (17)$$

The total inference latency $T_{i,j}$ for mobile user u_i is:

$$T_{i,j} = T_{i,j}^{comm} + T_{i,j}^{LoRA} + T_{i,j}^{inf}. \quad (18)$$

C. Deployment Costs

In addition to inference latency, we also focus on the LLM deployment costs. Deployment costs are directly determined by the number of edge servers rented and the storage capacity allocated on them. More servers and larger storage capacity result in higher deployment costs. Let $c_{j,l}$ denote the costs of deploying the base model f_l on edge server s_j , and $c_{j,h}$ denote the costs of deploying the LoRA matrix r_h on edge server s_j . Therefore, the overall LLM deployment costs are:

$$DC = \sum_{s_j \in S} \sum_{f_l \in F} c_{j,l} y_{j,l} + \sum_{s_j \in S} \sum_{r_h \in R} c_{j,h} x_{j,h}. \quad (19)$$

D. Problem Formulation

Our goal is to deploy LLMs to edge servers at the lowest possible deployment costs while minimize average inference latency. Let α denote the LLM service vendor's priority for lowering users' average inference latency. A large α indicates that the vendor is inclined to lower average inference latency for its users at higher deployment costs, and vice versa. Based on the inference latency and deployment costs models

introduced above, LECPL can be mathematically formulated as follows:

$$LECPL : \min_{y,x} \alpha \frac{\sum_{u_i \in U} T_{i,j}}{|U|} + (1 - \alpha) DC$$

s.t. (1) - (7),

where the expressions for $T_{i,j}$ and DC are given in (18) and (19), respectively. The proposed problem is NP-hard. We omit the proof due to space limitations.

IV. ALGORITHM DESIGN

In this section, we present EdgeColl to solve the LECPL. Since LECPL is an integer programming problem with a dependency between the deployment of base models and LoRA matrices, we decompose LECPL into two subproblems, i.e., the base model deployment (BMD) problem and the LoRA matrix deployment (LMD) problem. EdgeColl designs the BMD and the LMD algorithm to sequentially address these two subproblems.

A. BMD Algorithm

Given that edge servers deploying base models need to provide low-latency services to users, and each base model contains a large number of parameters, it is necessary to ensure that users requesting base model f_l are covered by at least one edge server deploying base model f_l . Moreover, to minimize deployment costs while covering users, base models should be deployed on as few edge servers as possible. Therefore, the BMD problem can be formally expressed as follows:

$$P1 : \min_y \alpha \frac{\sum_{u_i \in U} T_{i,j}^{base}}{|U|} + (1 - \alpha) \sum_{s_j \in S} \sum_{f_l \in F} c_{j,l} y_{j,l}$$

s.t. (1), (4), (5), (6), (7),

$$\sum_{f_l \in F} D_l^{base} y_{j,l} \leq q_j, \quad \forall s_j \in S,$$

where $T_{i,j}^{base} = T_{i,j}^{comm} + T_{i,j}^{inf}$. $P1$ aims to determine which base model f_l should be deployed on which edge server s_j , and that user request $\delta_{j,i}$ should be sent to which edge server s_j .

We design a base model deployment (BMD) algorithm to solve $P1$, as detailed in Algorithm 1. First, each (u_i, f_l) is treated as an element of the universal set $G = \{(u_i, f_l) | u_i \in U, f_l \in F\}$ (Lines 1-3). Next, we construct the set $e_{j,l} = (s_j, f_l)$, where each $e_{j,l}$ covers a finite set of (u_i, f_l) , i.e., $e_{j,l} = \{(u_i, f_l) | s_j \text{ covers } u_i\} \subseteq G$ (Lines 5-6). All sets $e_{j,l}$ together form the collection $E = \{e_{j,l} | s_j \in S, f_l \in F\}$ (Lines 7). The goal is to select the minimum number of subsets $e_{j,l}$ from E to cover all elements in G . To improve algorithm efficiency, we maintain a max heap where the set $e_{j,l}$ covers the largest number of (u_i, f_l) per unit deployment cost (Lines 9-10). Algorithm 1 runs multiple rounds. In each round, the top element $\tilde{e}_{j,l}$ is first popped from the heap (Line 12). We then perform a storage capacity evaluation. If $\tilde{e}_{j,l}$ does not meet the requirements, it will be discarded immediately and the next

round begins (Lines 13-15). Subsequently, a lazy evaluation is performed to determine whether $\tilde{e}_{j,l}$ should be re-pushed to the max heap (Lines 16-19). If both evaluations are passed, f_l is deployed on s_j , and the storage capacity of s_j is updated (Lines 20-21). Users covered in each round are marked, and the process continues until all users are covered (Line 22).

Algorithm 1 BMD algorithm

```

1: for  $u_i \in U$  do
2:   Build  $G = \{(u_i, f_l) | u_i \in U, f_l \in F\}$ ;
3: end for
4: for  $s_j \in S$  do
5:   Calculate which  $u_i$  are covered by  $s_j$ ;
6:   Build  $e_{j,l} = (s_j, f_l) = \{(u_i, f_l) | s_j \text{ covers } u_i\}$ ;
7:   Build  $E, E \leftarrow E \cup \{e_{j,l}\}$ ;
8: end for
9: Establish a max heap,  $MaxHeap$ , based on the number
   of  $(u_i, f_l)$  each  $e_{j,l}$  can cover per unit deployment costs;
10:  $MaxHeap.push(e_{j,l})$ ;
11: for  $MaxHeap$  not empty and  $G$  not empty do
12:    $\tilde{e}_{j,l} = (\tilde{s}_j, \tilde{f}_l) = MaxHeap.pop$ ;
13:   if  $\tilde{q}_j \leq D_l^{Base}$  then
14:     Back to Line 11;
15:   end if
16:   if  $|\tilde{e}_{j,l}|$  less than before entering the  $MaxHeap$  then
17:      $MaxHeap.push(\tilde{e}_{j,l})$ ;
18:     Back to Line 11;
19:   end if
20:    $y_{j,l} \leftarrow 1$ , i.e.,  $\tilde{f}_l$  is deployed on  $\tilde{s}_j$ ;
21:    $\tilde{q}_j = \tilde{q}_j - \tilde{D}_l^{Base}$ ;
22:   Remove  $(u_i, f_l) \in \tilde{e}_{j,l}$  from  $G$ ;
23: end for

```

B. LMD Algorithm

Since Algorithm 1 has determined the deployment decisions y^* of the base models, for the LMD problem, it is only necessary to deploy the LoRA matrices within the remaining storage capacity on the edge servers. When deploying, we need to ensure that the LoRA matrix is transferred to the edge server serving the user as quickly as possible, while considering the deployment costs of the LoRA matrices. Therefore, the LMD problem can be formally expressed as:

$$P2: \min_x \alpha \frac{\sum_{u_i \in U} T_{i,j}^{LoRA}}{|U|} + (1 - \alpha) \sum_{s_j \in S} \sum_{r_h \in R} c_{j,h} x_{j,h}$$

s.t. (2),

$$\sum_{r_h \in R} D_h^{LoRA} x_{j,h} \leq q_j - \sum_{f_l \in F} D_l^{Base} y_{j,l}^*, \quad \forall s_j \in S.$$

We design a LoRA matrix deployment (LMD) algorithm to solve $P2$. In Algorithm 2, we deploy each LoRA matrix sequentially. We check the storage capacities of all edge servers to ensure that the size of LoRA matrix to be deployed does not exceed the storage capacity (Lines 1-5). For each

LoRA matrix, we first relax the original mathematical model into a linear programming problem and obtain the optimal fractional solution of the LP as a benchmark (Line 6). Next, we filter the obtained fractional solutions by retaining the nearest neighbor assignments for the base model server that requested the LoRA matrix. We proportionally amplify the deployment volume of the LoRA matrix to restore the feasibility of the constraints, thereby obtaining feasible nearest neighbor fractional solutions (Line 7). Finally, rounding is performed to convert the nearest neighbor fractional solution to an integer solution (Line 8). Each time a LoRA matrix is deployed, the server storage capacity needs to be updated (Line 9).

Algorithm 2 LMD algorithm

```

1: for  $r_h \in R$  do
2:   Build a collection of edge servers  $Candidate$  that can
   deploy  $r_h$ ;
3:   if  $D_h^{LoRA} \leq q_j$  then
4:      $Candidate \leftarrow Candidate \cup \{s_j\}$ ;
5:   end if
6:   Obtain fractional solution under relaxed conditions;
7:   Filter fractional solution to obtain nearest neighbor
   fractional solution;
8:   Round nearest neighbor fraction to integer solution;
9:   Update server storage capacity;
10: end for

```

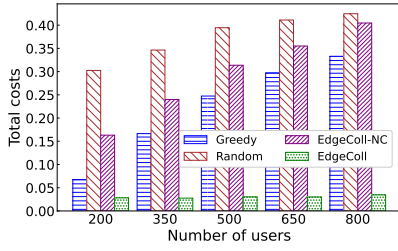
V. EXPERIMENT

A. Experiment Settings

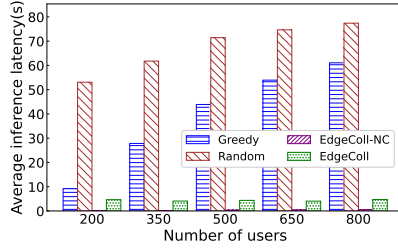
The experiments are conducted on the widely-used EUA dataset [9]. In the experiment, we set the coverage range of each edge server between 200m and 250m. The storage capacity is set between 320 and 420 GB. We assume that each edge server is equipped with RTX 4090 GPUs, with each having a computational speed of 82.6 TFLOPs/s and a memory bandwidth of 1008 GB/s. We select 100 personalized LLMs, all of which are fine-tuned using LoRA based on the three base models: Llama2-70B (130GB) [4], Qwen2.5-72B (134GB) [5], and PaLM-62B (115GB) [21]. The size of LoRA matrix ranges from 50 to 200MB. Each user randomly requests a personalized LLM, and the lengths of the input and output sequences are randomly generated within the range of 256 to 512 tokens. We vary the following parameters to emulate different LLM edge deployment scenarios: (1) number of mobile users, (2) number of edge servers, and (3) storage capacity of edge server.

B. Benchmark Approaches

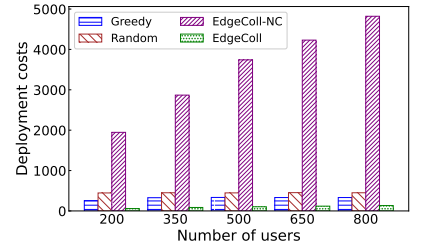
In the experiment, we compare EdgeColl with the following benchmarks: (1) Greedy algorithm: Deploy the entire LLM greedily on each edge server to minimize inference latency. (2) Random algorithm: Randomly deploy the entire LLM on each edge server. (3) EdgeColl-NC: Under the EdgeColl framework, minimize the inference latency while ignoring LLM deployment costs.



(a) Total costs vs. # of mobile users

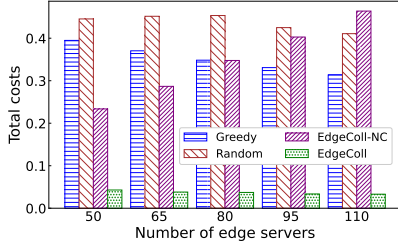


(b) Inference latency vs. # of mobile users

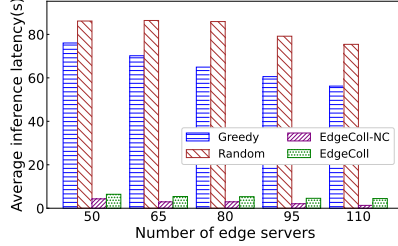


(c) Deployment costs vs. # of mobile users

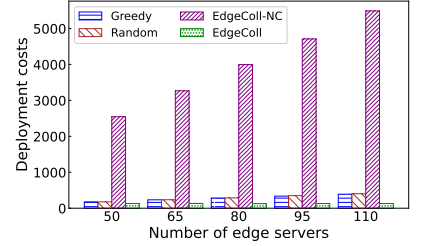
Fig. 4: Impact of mobile users.



(a) Total costs vs. # of edge servers

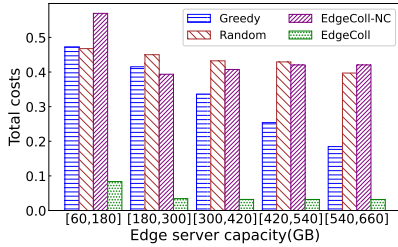


(b) Inference latency vs. # of edge servers

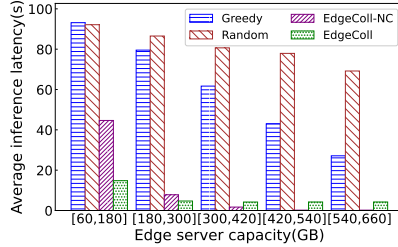


(c) Deployment costs vs. # of edge servers

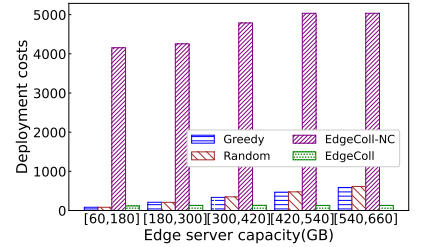
Fig. 5: Impact of edge servers.



(a) Total costs vs. Storage capacity



(b) Inference latency vs. Storage capacity



(c) Deployment costs vs. Storage capacity

Fig. 6: Impact of storage capacity.

C. Experiment Results

Impact of mobile users. Figs. (4a), (4b) and (4c) compare the total costs, average inference latency, and deployment costs of each approach under varying numbers of users. When the number of users changes, the average total costs of EdgeColl are 81.7% lower than those of Greedy, 91.9% lower than those of Random, and 88.9% lower than those of EdgeColl-NC. Additionally, as the number of users increases, the total costs of each approach show an upward trend. The reasons for the increase in EdgeColl and EdgeColl-NC are that the increase in the number of users requires the deployment of models on more servers, leading to higher deployment costs. However, for Greedy and Random, the limited server storage prevents providing low-latency services to the growing number of users, resulting in higher average inference latency.

Impact of edge servers. Figs. (5a), (5b) and (5c) show the impact of edge servers. When the number of edge servers changes, the average total costs of EdgeColl are 89.5% lower than those of Greedy, 91.6% lower than those of Random, and 88.4% lower than those of EdgeColl-NC. Unlike other approaches, as the number of servers increases, the total costs and average inference latency of EdgeColl always decrease

while deployment costs remain virtually unchanged. This is because more edge servers with sufficient storage and close proximity to users can deploy base models and LoRA matrices.

Impact of storage capacity. Figs. (6a), (6b) and (6c) demonstrate the impact of storage capacity. The average total costs of EdgeColl are 90.7% lower than those of Greedy, 90.3% lower than those of Random, and 86.9% lower than those of EdgeColl-NC. As storage capacity increases, the total costs and average inference latency of each approach decrease. With larger storage capacity, more models can be deployed in user-dense areas, thereby avoiding the need to rely on distant servers to satisfy user demand.

VI. RELATED WORK

Edge deployment of LLMs has been receiving increasing attention. Liu et al. [14] performed popularity modeling based on the probability of user requests for different models and prioritized caching high-popularity LLMs on edge servers to improve model cache hit rates under limited storage space. Lin et al. [22] employed model quantization to reduce model size and facilitate the deployment of more LLMs at the edge. Zhang et al. [13] proposed a distributed deployment

method that divides LLMs into layer-based shards distributed across edge and cloud servers, enabling collaborative inference without model quantization or compression. This approach overcomes the storage limitation of a single edge server. However, these studies focus mainly on LLM deployment while overlooking the redundant cache generated during the process. Qu et al. [23] decomposed LLMs into shared and specific parameter blocks. Since shared blocks are common across multiple LLMs, caching them once on an edge server can serve multiple model requests and greatly improve storage efficiency. However, they mainly focus on inference latency while overlooking the deployment costs of LLMs. As a comparison, EdgeColl takes both base models and LoRA matrices as deployment objects and jointly optimizes inference latency and deployment cost.

VII. CONCLUSION

In this paper, we proposed EdgeColl, an efficient LLM edge collaboration deployment strategy with LoRA. We formulated the LLM edge collaboration deployment problem and proposed a base model deployment strategy and a LoRA deployment strategy to solve the proposed problem. Experiments based on widely used real-world datasets validated the performance of EdgeColl.

REFERENCES

- [1] A. Zhang, Y. Deng, et al., “Large language model powered agents for information retrieval,” in *ACM SIGIR*, 2024, pp. 2989–2992.
- [2] S. Yang, H. Zhao, et al., “Zhongjing: Enhancing the chinese medical capabilities of large language model through expert feedback and real-world multi-turn dialogue,” in *AAAI*, vol. 38, 2024, pp. 19 368–19 376.
- [3] L. Chen, O. Sinavski, et al., “Driving with llms: Fusing object-level vector modality for explainable autonomous driving,” in *IEEE ICRA*, 2024, pp. 14 093–14 100.
- [4] H. Touvron, T. Lavril, et al., “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [5] A. Yang, B. Zhang, et al., “Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement,” *arXiv preprint arXiv:2409.12122*, 2024.
- [6] G. Qu, Q. Chen, et al., “Mobile edge intelligence for large language models: A contemporary survey,” *IEEE Communications Surveys & Tutorials*, pp. 1–1, 2025.
- [7] E. J. Hu, Y. Shen, et al., “Lora: Low-rank adaptation of large language models,” *ICLR*, vol. 1, no. 2, p. 3, 2022.
- [8] N. Su, C. Hu, et al., “Titanic: Towards production federated learning with large language models,” in *IEEE INFOCOM*, 2024, pp. 611–620.
- [9] L. Zhou, Z. Xu, et al., “Chasing common knowledge: Joint large model selection and pulling in mec with parameter sharing,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 3, pp. 437–454, 2025.
- [10] W. Gong, L. Yuan, et al., “Multiprotocol backscatter for personal iot sensors,” in *ACM CoNEXT*, 2020, pp. 261–273.
- [11] L. Yuan, C. Xiong, et al., “Embracing self-powered wireless wearables for smart healthcare,” in *IEEE PerCom*, 2021, pp. 1–7.
- [12] Y. Ding, C. Niu, et al., “Enhancing on-device llm inference with historical cloud-based llm interactions,” in *ACM SIGKDD*, 2024, pp. 597–608.
- [13] M. Zhang, X. Shen, et al., “Edgeshard: Efficient llm inference via collaborative edge computing,” *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13 119–13 131, 2024.
- [14] Z. Liu, H. Du, et al., “Joint model caching and resource allocation in generative ai-enabled wireless edge networks,” in *IEEE WCNC*, 2025, pp. 1–6.
- [15] M. Xu, D. Niyato, et al., “Joint foundation model caching and inference of generative ai services for edge intelligence,” in *IEEE GLOBECOM*, 2023, pp. 3548–3553.
- [16] L. Wang, S. Chen, et al., “Parameter-efficient fine-tuning in large language models: A survey of methodologies,” *Artificial Intelligence Review*, vol. 58, no. 8, p. 227, 2025.
- [17] X. Zhang, J. Nie, et al., “Beyond the cloud: Edge inference for generative large language models in wireless networks,” *IEEE TWC*, vol. 24, no. 1, pp. 643–658, 2024.
- [18] W. Kwon, Z. Li, et al., “Efficient memory management for large language model serving with pagedattention,” in *SOSP*, 2023, pp. 611–626.
- [19] A. Agrawal, N. Kedia, et al., “Taming throughput-latency tradeoff in llm inference with sarathi-serve,” in *USENIX OSDI*, 2024, pp. 117–134.
- [20] F. Yang, Z. Wang, et al., “Efficient deployment of large language model across cloud-device systems,” in *IEEE SoCC*, 2024, pp. 1–6.
- [21] A. Chowdhery, S. Narang, et al., “Palm: Scaling language modeling with pathways,” *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023.
- [22] J. Lin, J. Tang, et al., “Awq: Activation-aware weight quantization for on-device llm compression and acceleration,” *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.
- [23] G. Qu, Z. Lin, et al., “Trimcaching: Parameter-sharing ai model caching in wireless edge networks,” in *IEEE ICDCS*, 2024, pp. 36–46.